Software Cloud Computing 2026
Dr. John Zaki
Mohamed Abdelsatar
Jessica Ehab
Donia Ali

Mini-Jira on AWS

Deadline: 22/5/2026 at 11:59PM

Description: You will build and host a lightweight team task-management web application (think of it as a stripped-down Jira or Trello) fully running on AWS. The application supports multiple teams inside a company, where a manager assigns tasks to specific employees on specific teams, and each team only sees its own work. Beyond CRUD, the system uses event-driven AWS services (SNS, SQS, EventBridge), a Lambda-based image pipeline, and CloudWatch dashboards for monitoring — so the architecture genuinely exercises the topics covered in lectures. The app must be deployed in a high-availability setup across at least two Availability Zones, fronted by an Application Load Balancer and CloudFront. All persistence is on DynamoDB; uploaded images live in S3 and are processed by Lambda.

Functional Requirements

Roles and teams

The system has three role types and supports an arbitrary number of teams (e.g., Frontend, Backend, QA, DevOps). Every employee belongs to exactly one team. A manager is not bound to a single team — managers have visibility across the entire company.

- Manager — creates projects, creates tasks, assigns tasks to any employee on any team, sees all tasks across all teams, sees per-team dashboards.
- Employees — sees only the tasks assigned to their own team, can update status of tasks assigned to them, can comment and attach files.
- Admin (optional, can be merged with Manager) — creates teams and adds users to teams.

Task lifecycle

- Manager creates a task: title, description, priority, deadline, assignee, team, optional image attachment.
- Status flow: To Do → In Progress → In Review → Done.
- Comments thread on each task.
- File/image attachments stored in S3 and resized by a Lambda function on upload.
- Audit log of status changes (who moved it, when). Team isolation (important) Tasks must be filtered by the team on the server side, not just hidden in the UI. An employee on the Backend team must not be able to fetch a Frontend team task even by guessing its ID. Use a Global Secondary Index on (teamId) in DynamoDB and enforce the team check in every API handler. The manager bypasses this filter and sees everything.

CRUD coverage

• Create / Read / Update / Delete on Tasks.
• Create / Read / Update / Delete on Projects.
• Create / Read on Comments.
• Image upload, replacement (keeping old versions in S3), and deletion alongside the task.

Requirements:

- **Technology Stack:** Choose any JavaScript stack for developing the web application. Examples include MEAN (MongoDB, Express.js, AngularJS, Node.js), MERN (MongoDB, Express.js, React, Node.js), or Typescript Stack: NestJs & NextJs, or any other stack (you will use DynamoDB instead of Mongo).

- **CRUD Operations:** Implement Create, Read, Update, and Delete operations on Tasks, Projects, and Comments. Tasks have at minimum: title, description, status, priority, deadline, assignee, team, and an optional image attachment.

- **Role-Based Access (Manager / Employee / Teams):** The system must support three role types — Manager, Employee, and optionally Admin — and an arbitrary number of teams (e.g., Frontend, Backend, QA). Every employee belongs to exactly one team. The manager can assign tasks to any employee on any team and can view all tasks across the company. Employees can only view and modify tasks belonging to their own team. Filtering must be enforced server-side, not just hidden in the UI.

- **Demo Scenario (must work on demo day without code changes):** Manager Ali creates Task A and assigns it to Sara on the Frontend team, and creates Task B and assigns it to Omar on the Backend team. When Sara logs in, she sees only Task A. When Omar logs in, he sees only Task B. When Ali logs back in as manager, he sees both tasks and can filter by team.

- **Authentication:** Use AWS Cognito to manage users, sign-in, sign-up, and to store role and team membership. Tokens issued by Cognito must be validated by your backend on every request.

- **High Availability Architecture:** Design the architecture for high availability by deploying the application across multiple EC2 instances in different Availability Zones within a region, behind an Application Load Balancer and an Auto Scaling Group. Include CloudFront for faster delivery of your application.

- **AWS SDK Usage:** Use AWS SDK for JavaScript to interact with the AWS services programmatically from your application backend (DynamoDB, S3, SNS, SQS, CloudWatch custom metrics, etc.).

- **DynamoDB Integration:** Use DynamoDB to store and manage all application data. Design tables for Users, Teams, Projects, Tasks, and Comments. The Tasks table must include at least one Global Secondary Index on teamId and one on assigneeId to support team-scoped queries.

- **S3 Image Upload:** Implement functionality to upload images for newly created tasks. Store the images in an S3 bucket. Ensure that the images are displayed or deleted when the corresponding task is retrieved or deleted. Allow updating the image associated with a task and ensure that both old and new versions are retained.

- **Link Images to Tasks:** Associate the uploaded images stored in the S3 bucket with their corresponding tasks in the DynamoDB Tasks table.

- **Lambda Function (Image Resize):** Create a Lambda function that is triggered only upon the creation of newly added tasks. This function is responsible for resizing images uploaded to the S3 bucket. If needed, you may create an additional bucket for the resized images.

- **Event-Driven Notifications (SNS + SQS):** When a manager assigns a task to an employee, publish an event to an SNS topic. The topic must fan out to (a) an email subscription that notifies the assignee, and (b) an SQS queue that is drained by a worker Lambda. The worker Lambda writes an entry to an activity log and publishes a custom CloudWatch metric (e.g., TasksAssignedPerTeam).

- **Scheduled EventBridge Rule:** Create an EventBridge scheduled rule that runs every day at 9:00 AM. The rule must trigger a Lambda that scans the Tasks table for tasks due that day and sends each assignee a digest email via SNS.

- **CloudWatch Monitoring:** Build a CloudWatch dashboard with at least four widgets: tasks created per day, tasks closed per day per team, average time-to-close, and EC2 CPU utilization. Configure at least one CloudWatch alarm (e.g., overdue tasks exceeding a threshold) that publishes to an SNS topic.

- **UI / UX:** The frontend must look polished. Use a modern UI library (Tailwind + shadcn/ui, Material UI, Chakra, or Ant Design). Include a Kanban board view (To Do / In Progress / In Review / Done) with drag-and-drop, a task detail modal with comments, loading and empty states, and proper error toasts.

AWS Architecture:

Below is the full list of AWS services your deployment must use. Each one has a specific role in the system — be ready to justify each during the demo. The diagram you submit must show how these components connect across two Availability Zones.

Service	Role in the system
EC2 (Auto Scaling Group)	Hosts the Node.js backend across at least 2 Availability Zones.
Application Load Balancer	Distributes traffic across EC2 instances and runs health checks.
CloudFront	CDN in front of the ALB for low-latency delivery of the app and static assets.
DynamoDB	Stores Users, Teams, Projects, Tasks, and Comments. GSIs on teamId and assigneeId.

Service	Role in the system
S3 (originals bucket)	Stores task image attachments uploaded by users. Old versions are retained on update.
S3 (resized bucket)	Stores thumbnails generated by the image-resize Lambda.
Lambda — Image Resize	Triggered by S3 PUT events on the originals bucket. Writes thumbnails to the resized bucket.
Lambda — Assignment Worker	Drains the SQS queue, writes activity logs, publishes CloudWatch custom metrics.
Lambda — Daily Digest	Triggered by EventBridge at 9:00 AM. Scans tasks due today and sends digest emails via SNS.
SNS	Fan-out for task-assignment events: notifies the assignee by email and feeds the SQS queue.
SQS	Buffers assignment events for the worker Lambda. Decouples the API from background processing.
EventBridge	Scheduled rule that runs the daily digest Lambda.
Cognito	User pool for sign-up / sign-in. Stores role and teamId as user attributes.
CloudWatch	Custom metrics, dashboard, and alarms (e.g., overdue tasks above threshold).
IAM	Least-privilege roles for EC2, each Lambda, and the application identity.
VPC + Subnets	Public subnets for the ALB; private subnets for EC2; NAT gateway for outbound traffic.

Deliverables:

- Project GitHub Repo Link
- The Repo Must Include .md file contains :
    - Detailed architecture diagram illustrating the high availability setup ***(The detailed architecture must be drawn using AWS standard icons outlined in this link:*** *https://aws.amazon.com/architecture/icons/*, ***either through a PowerPoint presentation or one of the associated tools such as Lucidchart..)***
    - Working public link to the deployed web application (the CloudFront distribution URL). Clicking the link must open the live website directly without any additional configuration.
    - Demo Video For the Project.

Submission:

- Submit the form (https://docs.google.com/forms/d/e/1FAIpQLSdOo4eouZwbf-dNVfwFvraYxGZx6TTdsflE-DISRQX3jWTPkg/viewform).

🛑Remember **NOT** to terminate any instances or used resources after your submission. Instead, simply stop them. Terminating any resources will result in a loss of progress or data, which will be considered as zero.

***Note:** It is important to keep in mind the limitations of the AWS Free Tier to avoid exceeding budget constraints. For instance, the maximum free tier EBS storage is 30GB, and instances can run up to 750 hours per month (same applies to an ALB service).
Therefore, remember to stop instances when not in use to avoid unnecessary charges. ***Also, always review the pricing of each service before launching to ensure compatibility with free tier usage limits**.